

1. 单一职责原则（SRP）

原则含义：一个类或模块应该有且只有一个引起它变化的原因，即职责要单一。

在 StuffHub 中的体现与应用：

我们将系统拆分为四个核心模块：

- 员工信息管理：只负责员工基本信息的增删改查、状态管理；
- 薪资管理：只负责薪资项配置、工资条生成、历史查询；
- 查询统计：只负责多条件组合查询、汇总与导出；
- 系统管理：只负责权限分配、备份恢复、操作日志。

每个模块的职责边界非常明确。例如，当《工资支付暂行规定》发生变化，需要修改薪资计算逻辑时，只会改动薪资管理模块，员工信息模块完全不受影响。

实践示例：在 Python 代码中，`EmployeeService` 类只包含 `add_employee`、`update_status` 等方法，绝不包含计算工资或生成报表的代码。这样做避免了“万能类”，让新开发人员（如文档所说 3 天内上手）能快速定位和修改功能。

2. 开闭原则（OCP）

原则含义：软件实体应对扩展开放，对修改封闭。即新增功能应通过增加新代码实现，而非修改已有代码。

在 StuffHub 中的体现与应用：

我们 SAD 文档第 9.1 节关键决策四明确说明：暂不实现考勤、绩效等模块，但“预留接口便于未来扩展”。这一决策直接体现了 OCP。

系统采用 Flask 的蓝图（Blueprint）来组织模块，未来加入考勤模块只需新增一个 `attendance` 蓝图和对应的服务类，注册到应用中即可，无需修改员工、薪资等模块的代码。

查询统计模块也需要遵循 OCP：它通过动态拼接参数化查询来支持新筛选条件，未来要增加“按学历统计”的功能，只需在查询接口中增加一个可选参数，后端再增加一条安全的 SQL 片段，不影响已有统计功能。

实践示例：定义薪资项计算策略时，可抽象一个 `SalaryItemCalculator` 接口，基本工资、奖金、扣款分别实现该接口。当需要新增一种“股权激励”薪资项时，只需新增一个实现类，通过配置文件注入，不修改已有的计算逻辑。

3. 利斯科夫替换原则（LSP）

原则含义：子类必须能够完全替换其基类，并且替换后程序的行为仍符合预期。

在 **StuffHub** 中的体现与应用：

文档 4.3 节定义了三种角色：管理员、部门管理者、普通员工。它们都可以抽象为基类 **User**，其中包含 `get_viewable_payroll()`、`can_edit_employee()` 等行为方法。管理员子类实现全部权限，部门管理者子类只有本部门查询和统计权限，普通员工只能查看自己的信息。在权限校验的代码中，只要传入的是 **User** 类型，不需要判断具体是哪个子类，程序就能正确执行。

实践示例：薪资查询接口 `view_payroll(user: User, month)` 是根据传入的 **user** 对象动态判断可展示的工资条范围。如果某天新增“审计员”角色，它继承自 **User**，只要实现了正确的约束方法，就可以无缝替换进系统，原有接口不用修改，这正符合 LSP。

4. 德米特法则（LoD）

原则含义：又称最少知识原则，一个对象应对其他对象有尽可能少的了解，只与直接的朋友通信。

在 **StuffHub** 中的体现与应用：

SAD 文档 5.2 节明确规定了三层 B/S 架构：表示层只能调用业务逻辑层，业务逻辑层只能调用数据访问层。表示层完全不接触数据库，避免了前端代码直接发 SQL 的混乱。模块间的依赖也遵循了 LoD（5.3 节）：薪资管理模块依赖员工信息模块，但它只应调用员工信息模块对外暴露的“获取在职员工列表”这类服务方法，而不是去直接操作 **employee** 表，也不应知道员工信息模块内部使用了 ORM 还是原生 SQL。

实践示例：生成工资条时，**PayrollService** 内部通过 `employee_service.get_active_employees()` 获取员工，绝不引用 **Employee** 模型类或拼接 SQL。这样，当员工数据表结构改变时，只需修改 **EmployeeService** 内部，不会波及 **PayrollService**，实现了真正的松耦合。

5. 依赖倒转原则（DIP）

原则含义：高层模块不应该依赖低层模块，两者都应该依赖其抽象：抽象不应该依赖细节，细节应该依赖抽象。

实践示例：系统管理模块需要记录日志，可以定义 **ILogRepository** 接口，当前使用 **MySQLLogRepository** 将日志写入 **log** 表。未来如果要远程发送日志到 Syslog 服务器，只需新建一个 **SyslogRepository** 实现，将对象注入给系统管理服务，完全符合 DIP，不会影响调用方的代码。

6. 合成复用原则（CARP）

原则含义：尽量使用对象组合，而不是继承来达到复用的目的。组合比继承更灵活，耦合度更低。

在 StuffHub 中的体现与应用：

SAD 文档 5.3 节的模块依赖关系已经暗示了组合：薪资管理模块依赖员工信息模块和系统管理模块，而不通过继承获得这些功能。我们可以把这种依赖直接设计为组合：

PayrollService 类中组合了 **EmployeeService**（负责获取员工列表）和 **PermissionService**（负责校验权限）的实例，通过构造函数注入。它不需要继承 **EmployeeService** 来获得员工数据，否则会导致“为复用而强加父子关系”的紧耦合。同样，查询统计模块组合了 **EmployeeService** 和 **PayrollService**，而不是通过多重继承来完成统计。

总结

上述六个原则在 StuffHub 系统中是相辅相成的：

- **SRP** 保证了模块的纯净性；
- **OCP** 让系统可持续演进（预留考勤扩展）；
- **LSP** 保障角色继承体系的正确性；
- **LoD** 严格控制跨层访问；
- **DIP** 将具体实现隔离在接口之后；
- **CARP** 通过组合灵活集成各个服务。

这些原则共同支撑起文档强调的“可维护性、可扩展性、低耦合”架构目标，正是项目能够控制成本、快速交付并安全运行的关键所在。